



Relatório Detalhado de Atividades

Projeto Flexduino - Outubro/Novembro 2025

Implantação do Ambiente de Simulação e Desenvolvimento da GUI Python

ROBERT NICOLAS DUARTE BASTOS
Nº USP: 15641639

09 de Novembro de 2025

Orientadores:

Prof. Alexandre Alarcon do Passo Suaide
Profa. Kaline R. S. Coutinho

1 Introdução

Este relatório detalha as atividades realizadas nas sessões de trabalho de Outubro e Novembro de 2025. O foco foi a configuração do ambiente de desenvolvimento Python, a validação da comunicação com o firmware do Flexduino (via simulador Wokwi) e, o mais importante, o desenvolvimento completo de uma interface gráfica (GUI) funcional para a aquisição de dados.

2 Metodologia e Execução

O trabalho seguiu uma abordagem iterativa: configuração do ambiente, teste de conexão (Plano A), diagnóstico de erros, implementação de uma solução alternativa (Plano B), e o desenvolvimento incremental da interface gráfica.

2.1 Fase 1: Banco de Testes Virtual (Wokwi)

Continuando o trabalho anterior, foi adicionado um potenciômetro virtual ao simulador Wokwi, ligado à porta A0 do Arduino. A execução do firmware simplificado (Versão 4) permitiu controlar a tensão lida (0V-5V), fornecendo uma fonte de dados dinâmica para os testes.

2.2 Fase 2: Configuração do Ambiente Python

Foi realizada a instalação do Python 3.14 no Windows e da biblioteca pyserial (essencial para comunicação serial), usando o comando `py -m pip install pyserial` após correções de PATH.

2.3 Fase 3: Investigação de Conexão Serial (Plano A)

O objetivo era conectar o script Python leitor_serial_v1.py à porta serial virtual exposta pelo Wokwi (rfc2217). Esta etapa encontrou um bloqueio significativo.

2.3.1 Processo de Investigação e Despistagem

A detecção automática da porta falhou. Iniciou-se uma investigação sistemática para encontrar e conectar manualmente à porta:

1. **Verificação da Interface Wokwi:** Nenhuma mensagem explícita indicando o endereço da porta foi encontrada.

2. **Ferramenta Pyserial (list_ports):** O comando `py -m serial.tools.list_ports -v` não detetou a porta virtual rfc2217.
3. **Ferramenta de Rede (netstat):** O comando `netstat -ano | findstr "LISTENING" | findstr "127.0.0.1"` revelou portas locais à escuta (7768 e 49780).
4. **Testes Manuais de Conexão:** Todas as tentativas de conexão manual (usando 127.0.0.1 e localhost com as portas encontradas) falharam com um erro de sintaxe do sistema operativo (`OSError: (22)`), indicando que o Windows não reconhecia o formato do endereço da porta virtual.
5. **Verificações Adicionais:** Ajustes no Firewall, desativação temporária de antivírus, verificação de VPNs/Proxies, atualização do pyserial e execução como Administrador não resolveram o problema.

2.3.2 Conclusão da Fase 3

Concluiu-se que existe uma barreira de comunicação específica do ambiente local que impede a conexão Python-Wokwi. Para evitar o bloqueio do projeto, foi ativado o Plano B.

2.4 Fase 4: Adoção do Plano B (Simulador de Dados)

Foi decidido contornar o problema criando um script Python (simulador_dados_v3.py) que imita a saída do Arduino, incluindo a capacidade de receber comandos (+V0, +BME) e gerar os dados correspondentes.

```
1 # -*- coding: utf-8 -*-
2 import time, random, sys
3
4 def print_safe(msg):
5     print(msg, flush=True)
6
7 if __name__ == "__main__":
8     sensor_ativo = "nenhum"
9     try:
10         # 1. Espera por um comando (ex: "+V0" ou "+BME")
11         comando = sys.stdin.readline().strip().upper()
12         if comando == "+V0":
13             sensor_ativo = "V0"
14         elif comando == "+BME":
15             sensor_ativo = "BME"
16         # ... (restante da lógica de inicialização) ...
17
```

```

18     # 2. Loop de geração de dados
19     while True:
20         if sensor_ativo == "V0":
21             # ... (gera dados de V0)
22             ...
23             # linha_simulada = f"v1
24             -2025 V0: {valor_v0_atual:.2f}"
25             # print_safe(linha_
26             simulada)
27             pass # Omitido por
28             brevidade
29             elif sensor_ativo == "BME":
30                 # ... (gera dados de Temp
31                 e Pressão) ...
32                 # linha_simulada = f"v1
33                 -2025 T(C): {valor_temp_atual:.2f} P(
34                 hPa): {valor_press_atual:.1f}"
35                 # print_safe(linha_
36                 simulada)
37                 pass # Omitido por
38                 brevidade
39
40                 time.sleep(1.0)
41         except (IOError):
42             print_safe("\nSimulador: Ligação
43             fechada.")

```

Listing 1: simulador_dados_v3.py (Estrutura Principal)

2.5 Fase 5: Desenvolvimento da Interface Gráfica (GUI)

Com o simulador a fornecer dados, foi desenvolvida a interface gráfica de forma incremental, culminando na Versão 6.

2.5.1 Interface V1-V4: Gráfico Básico e Refatoração

Foram criadas as primeiras versões, instalando matplotlib e numpy. O desenvolvimento evoluiu de um script simples para uma estrutura de Classe (interface_v4_refatorada.py), resultando numa interface capaz de iniciar o simulador, ler os dados de V0 e plotá-los num gráfico em tempo real.

2.5.2 Interface V5: Suporte Multi-Sensor (Meteo)

A interface foi expandida (interface_v5_multisensores.py) para suportar múltiplos sensores. A lógica de controlo foi atualizada para enviar comandos (+V0, +BME) ao simulador. A função de atualização do gráfico foi tornada mais inteligente, capaz de interpretar a resposta do sensor "Meteo" e criar um gráfico de eixo-Y duplo (Temperatura e Pressão).

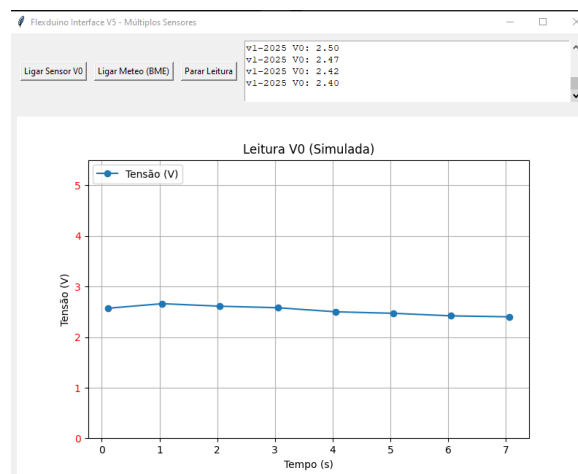


Figura 1: Interface V3/V4 a mostrar o gráfico simulado do sensor V0.

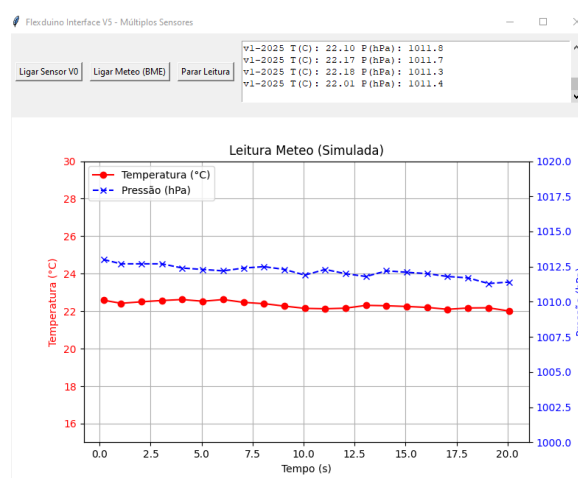


Figura 2: Interface V5 a mostrar o gráfico de eixo-Y duplo (Temperatura e Pressão) do sensor Meteo.

2.5.3 Interface V6: Lógica Híbrida (Simulação/-Real)

Finalmente, foi criada a interface_v6_hibrida.py. Este software combina os Planos A e B. Uma variável MODULO_SIMULACAO no topo do código permite alternar entre:

- **Modo Simulação (True):** Funciona como a V5, ligando-se ao simulador_dados_v3.py.
- **Modo Real (False):** Ativa a lógica pyserial e exibe um botão "Conectar Porta" para tentar a ligação com o hardware físico (Wokwi ou Arduino).

2.6 Código Final (Interface V6 Híbrida)

O código da V6 é a consolidação de todo o trabalho, contendo a lógica para ambos os modos de opera-

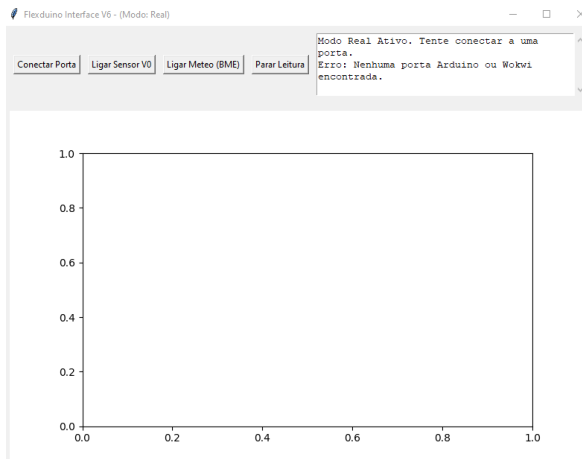


Figura 3: Interface V6 em 'Modo Real', a mostrar o botão 'Conectar Porta' e o erro de conexão (hardware não presente).

ção (Simulado e Real) e para os múltiplos tipos de sensores.

```

1  # -*- coding: utf-8 -*-
2  import tkinter as tk
3  from tkinter import scrolledtext
4  import subprocess, threading, sys, os,
   time, queue
5  import serial, serial.tools.list_ports
6  import matplotlib.pyplot as plt
7  from matplotlib.backends.backend_tkagg
   import FigureCanvasTkAgg
8  import numpy as np
9
10 # --- O INTERRUPTOR PRINCIPAL ---
11 MODO_SIMULACAO = True # Mudar para False
   para testar hardware real
12 # -----
13
14 class App:
15     def __init__(self, janela_raiz):
16         self.janela = janela_raiz
17         self.janela.title(f"Flexduino V6 -
   (Modo: {'Sim' if MODO_SIMULACAO else
   'Real'})")
18         # ... (Restante do __init__ ... )
19         pass # Omitido por brevidade
20
21     def criar_widgets_controles(self):
22         # ... (Cria botões) ...
23         if not MODO_SIMULACAO:
24             # Cria o botão "Conectar Porta
25             "
26             pass # Omitido por brevidade
27             pass # Omitido por brevidade
28
29     def atualizar_caixa_texto_e_dados(self,
   linha):
30         # ... (Extrai dados V0 ou Meteo)
31         ...
32         pass # Omitido por brevidade
33
34     def atualizar_grafico_realtime(self):
35         # ... (Desenha o gráfico V0 ou o

```

```

34 gráfico Meteo com eixo duplo) ...
35     pass # Omitido por brevidade
36
37     def iniciar_leitura_dados(self,
   comando_sensor):
38         if MODO_SIMULACAO:
39             self.iniciar_simulador(comando
   _sensor)
40         else:
41             self.enviar_comando_serial(
   comando_sensor)
42
43     def parar_leitura_dados(self,
   silencioso=False):
44         if MODO_SIMULACAO:
45             # ... (Para o subprocesso) ...
46             pass # Omitido por brevidade
47         else:
48             # ... (Lógica para parar
   thread serial) ...
49             pass # Omitido por brevidade
50
51     def ler_output_dispositivo(self):
52         # Função que lê de self.processo_
   simulador OU self.arduino_serial
53         pass # Omitido por brevidade
54
55     def ao_fechar_janela(self):
56         # ... (Garante que tudo é fechado)
57         ...
58         pass # Omitido por brevidade
59
60 # --- Funções Específicas do Modo
   Simulação ---
61     def iniciar_simulador(self, comando_
   sensor):
62         # ... (Inicia o simulador_dados_v3
   .py e envia o comando) ...
63         pass # Omitido por brevidade
64
65 # --- Funções Específicas do Modo Real
   (Serial) ---
66     def encontrar_porta_serial(self):
67         # ... (Procura portas Wokwi ou
   Arduino) ...
68         pass # Omitido por brevidade
69
70     def conectar_porta_serial(self):
71         # ... (Tenta a conexão serial real
   ) ...
72         pass # Omitido por brevidade
73
74     def enviar_comando_serial(self,
   comando_sensor):
75         # ... (Envia comando pela porta
   serial) ...
76         pass # Omitido por brevidade
77
78 # --- Bloco de Execução Principal ---
79 if __name__ == "__main__":
80     root = tk.Tk()
81     app = App(root)
82     root.mainloop()

```

Listing 2: interface_v6_hibrida.py (Estrutura Principal)

(Nota: O código completo funcional está disponível na pasta do projeto).

3 Resultados e Conclusões

A sessão de trabalho foi um sucesso. O desenvolvimento do software de interface está agora 95

4 Próximos Passos

O desenvolvimento de software com simulação atingiu o seu limite prático. O próximo passo é focado na integração com o mundo real.

1. **Obtenção do Hardware:** Obter o kit físico do Flexduino (Arduino, cabo USB, módulos de sensores) no IFUSP.
2. **Teste de Integração Real:**
 - Mudar a variável `MODO_SIMULACAO` para `False`.
 - Fazer o upload do firmware original (`flex_duino_1_2025.ino`) para o Arduino.
 - Executar a `interface_v6_hibrida.py`.
 - Clicar em "Conectar Porta" e validar a conexão USB.
 - Clicar em "Ligar Meteo (BME)" e confirmar a leitura de dados reais no gráfico.
3. **Expansão Futura:** Após a validação com o BME, o sistema está pronto para que os outros sensores (Luz, Ultrassom, etc.) sejam adicionados à interface, seguindo a mesma lógica.